DBZS
CODING-CHAT

ARCHITECTURE & CODE INTELLIGENCE BLUEPRINT

Runtime · Agenten · Modellintelligenz · Memory · GUI · Sicherheit

LEITIDEE
Vom lokalen LLM-Chat zur modularen, nachvollziehbaren Entwicklungsplattform.

Version 1.0 | 02. November 2026
Konsolidierte Spezifikation aus diesem Chat, bestätigten Architekturentscheidungen und bereitgestellten Projektunterlagen.

DESIGN-MOTTO / FUNCTION OVER DECORATION

Dieses Dokument ordnet die bisherigen Inhalte neu, schärft sie hinsichtlich Code-Intelligenz und übersetzt sie in ein grafisch lesbares technisches Zielbild. Es behauptet nicht, dass vorgeschlagene Komponenten bereits implementiert oder getestet sind.

00 / ÜBERBLICK

Executive Summary

Die zentrale Produktidee und die fünf bestätigten Architekturentscheidungen.

DBZS-CODING-CHAT soll ein lokales, modulares Entwicklungswerkzeug werden, das Codebasen nicht nur per Prompt bespricht, sondern sie strukturiert versteht, Änderungen kontrolliert vorbereitet, Ergebnisse überprüft und den gesamten Arbeitsweg nachvollziehbar macht.

DAS SYSTEM IN EINEM BLICK



Bestätigte Architekturentscheidungen

Bereich	Entscheidung
Agenten	Eigener DBZS Orchestrator + LangGraph für komplexe Workflows
Modell-Runtimes	Dynamisches Scheduling nach VRAM, RAM, CPU und sonstigen Ressourcen
API	Interne Services; API nur für externe Integrationen
Memory	Modulares Hybrid-Memory mit Embeddings und Reranking
Plattform	Von Anfang an plattformübergreifend

Wichtige Einordnung: Die GUI-Richtlinie stammt aus dem bereitgestellten DBZSCC2611-Designbrief. Sie wird als Gestaltungsleitlinie berücksichtigt; sie ändert nicht automatisch die Identität oder den Implementierungsstand des Coding-Chat-Repositories.

01 / KERNFÄHIGKEIT

Code-Intelligenz als Produktkern

Code Intelligence ist mehr als Codegenerierung: Sie verbindet statische Struktur, Laufzeitbeobachtung, Retrieval, Planung und Verifikation.

Zieldefinition

DBZS soll ein Projekt als zusammenhängendes System verstehen: Dateien, Module, Symbole, Aufrufbeziehungen, Abhängigkeiten, Tests, Konfigurationen und Git-Änderungen. Das Modell formuliert und begründet; deterministische Werkzeuge liefern überprüfbare Fakten.



Intelligenzschichten

Schicht	Aufgabe	Beispiel
1. Repository Awareness	Dateien, Projektstruktur, Sprachen, Build-Systeme erkennen	Welche Module bilden den Einstiegspunkt?
2. Syntax & Semantik	AST, Symbole, Imports, Typen und Referenzen extrahieren	Wo wird diese Funktion aufgerufen?
3. Code Graph	Abhängigkeiten und Beziehungen modellieren	Welche Komponenten sind von der Änderung betroffen?
4. Hybrid Retrieval	Textsuche, Symbolsuche und semantische Suche kombinieren	Finde ähnliche Implementierungen und relevante Tests.
5. Reasoning & Planning	Aufgabe zerlegen, Risiken und Änderungsumfang bestimmen	Welche Dateien müssen angepasst werden?
6. Controlled Editing	Kleine, überprüfbare Änderungen erzeugen	Patch statt unkontrolliertem Überschreiben
7. Verification	Tests, Lint, Typprüfung und Diff prüfen	Welche Akzeptanzkriterien sind nachweislich erfüllt?

Leitregel: LLMs dürfen Hypothesen und Vorschläge liefern. Dateipfade, Symbolreferenzen, Testresultate, Exit-Codes und Git-Diffs müssen aus realen Werkzeugergebnissen stammen.

02 / ENGINEERING LOOP

Der Code-Intelligence-Zyklus

Jede relevante Änderung folgt einem nachvollziehbaren, wiederholbaren Prozess.



1. Verstehen

- Workspace eindeutig identifizieren; Branch, Commit und Dirty State erfassen.
- Sprachen, Frameworks, Entry Points, Konfigurationen und Tests erkennen.
- Index und Code Graph inkrementell aktualisieren.

2. Belegen

- Relevante Dateien und Symbole mit Pfad- und Zeilenbezug abrufen.
- Abhängigkeiten und potenziell betroffene Tests ermitteln.
- Fakten, Annahmen und Unsicherheiten getrennt kennzeichnen.

3. Planen und Freigeben

- Änderungsplan mit Ziel, Dateien, Risiken und Akzeptanzkriterien erstellen.
- Bei unklarer oder folgenreicher Änderung gezielt nachfragen.
- Vor schreibenden oder destruktiven Aktionen den erforderlichen Approval Gate anwenden.

4. Ändern und prüfen

- Kleine Diffs erzeugen; bestehende Nutzeränderungen schützen.
- Formatierung, Lint, Typprüfung und passende Tests ausführen.
- Fehler anhand tatsächlicher Logs analysieren; Korrekturschleife begrenzen.

5. Liefern

- Diff, betroffene Dateien, Testbefehle und Ergebnisse zusammenfassen.
- Nicht ausgeführte Prüfungen klar als offen ausweisen.
- Evidence an Task, Workspace und Commit binden.

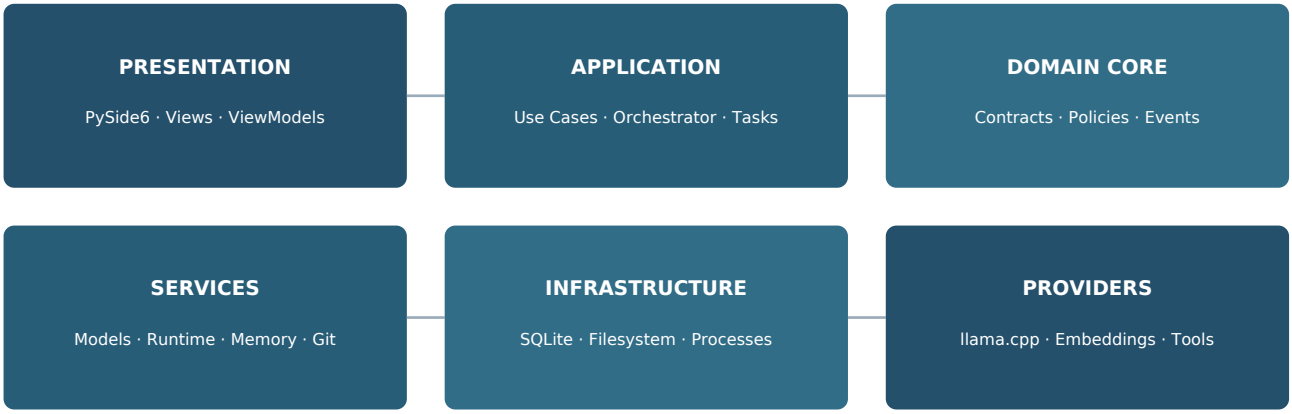
Abnahmekriterium: Eine Änderung ist erst abgeschlossen, wenn das gewünschte Artefakt vorliegt und der erreichbare Prüfstatus transparent dokumentiert wurde.

03 / SYSTEMARCHITEKTUR

Schichtenmodell und Verantwortlichkeiten

UI, Anwendung, Core und Infrastruktur bleiben voneinander getrennt.

LOGISCHE ARCHITEKTUR



Schichtverträge

Schicht	Darf	Darf nicht
Presentation	Zustand anzeigen, Eingaben erfassen, Services anstoßen	Direkt SQL ausführen oder Runtime-Prozesse starten
Application	Use Cases koordinieren, Policies anwenden, Tasks verwalten	An Provider-Implementierungen fest verdrahten
Domain Core	Verträge, Zustände, Ereignisse und Regeln definieren	PySide-, FastAPI- oder llama.cpp-spezifische Details kennen
Services	Fachliche Funktionen wie Registry, Scheduler, Retrieval bereitstellen	Unkontrolliert fremde Module umgehen
Infrastructure	Datenbank, Prozesse, Netzwerk und Dateisystem kapseln	Fachliche Entscheidungen treffen
Providers	Modelle, Tools und Backends konkret ausführen	Orchestrator oder Datenhoheit übernehmen

Architekturprinzip: Abhängigkeiten zeigen nach innen. Der Core bleibt möglichst frameworkneutral; Adapter verbinden ihn mit konkreten Bibliotheken und Betriebssystemfunktionen.

04 / RUNTIME

Dynamische Modell- und Ressourcensteuerung

Der Runtime Controller koordiniert Modelle, Prozesse und Ressourcen. Modellwahl und Rollenbelegung bleiben getrennt.

RUNTIME CONTROL PLANE



Ressourcenmodell

Signal	Berücksichtigung
VRAM	Freier und reservierter Speicher, Modellgewichte, KV-Cache, Batch/Context
RAM	Systemlast, Modell-Offload, Embedding/Reranking parallel
GPU	Gerät, Backend, Auslastung und Kompatibilität
CPU	Kerne, Threadbudget und parallele Tasks
Speicher / I/O	Modellpfad, Ladezeit, freier Speicher
Runtime State	Laufende Prozesse, aktive Sessions, Prioritäten

Scheduling-Regeln

- Vor jedem Start Ressourcen schätzen und reservieren; nach Start tatsächliche Werte und Health prüfen.
- GPU-/CPU-/Hybrid-Auswahl anhand von Modell, Hardware und Aufgabe treffen.
- Fallback nur kontrolliert und nur bei nachgewiesener Backend-Unterstützung.
- Idle-Unload und Cold-Swap konfigurierbar; aktive Aufgaben nicht überraschend unterbrechen.
- Port- und Prozessidentität eindeutig verwalten; fremde Prozesse nicht versehentlich beenden.

Wichtige technische Einschränkung: VRAM-Schätzungen sind Näherungen. Der Scheduler braucht Sicherheitsreserve, Runtime-Feedback und klare Fehlerpfade; eine statische VRAM-Zahl ist keine Startgarantie.

05 / MODELLINTELLIGENZ

Model Registry und Capability Resolution

Der Nutzer wählt primär eine Aufgabe oder Rolle – nicht einen Dateipfad.



Modellmetadaten und Fähigkeiten

Gruppe	Beispiele
Identität	ID, Name, Pfad, Hash, Familie, Architektur, Format, Quantisierung
Capabilities	Chat, Instruct, Coding, Agentic, Reasoning, Tool Calling, Vision, OCR
Spezialmodelle	Embedding, Reranker, Draft, Translation, Classification
Hardware	Backend, CPU/GPU-Kompatibilität, VRAM-/RAM-Schätzung, Context
Qualität	Benchmark, Speed, Quality, Last Used, Last Tested, Favorite
Abhängigkeiten	mmproj, Adapter, Draft-Modell, Runtime-Profil

MM-Projektor-Pairing

- Zuordnung anhand Metadaten, Modellfamilie, normalisiertem Dateinamen, Version und Ordnernähe.
- Pairing-Status sichtbar machen: bestätigt, wahrscheinlich, manuell bestätigt, ungeklärt, inkompatibel.
- Ungeklärte Paare nicht stillschweigend als kompatibel behandeln.

Task-aware Model Picker

Kandidaten werden zuerst nach Capability gefiltert, dann nach Hardwareeignung, bestätigten Tests, Benchmarkqualität, Ressourcenverbrauch und letzter erfolgreicher Nutzung sortiert. Der manuelle Dateipicker bleibt als Experten-/Notfallpfad erhalten.

Persistenzvorschlag: SQLite als führende Metadatenquelle; Tabellen für models, model_files, capabilities, pairs, benchmarks, hardware_profiles, runtime_profiles, role_assignments und model_roots.

06 / AGENTEN & WORKFLOWS

Eigener Orchestrator plus LangGraph

Der DBZS Orchestrator besitzt die globale Steuerung. LangGraph führt geeignete komplexe, zustandsbehaftete Workflows aus.

AGENT CONTROL FLOW



Verantwortungstrennung

DBZS Orchestrator	LangGraph
Intent Resolution und globale Task-Verwaltung	Graph-Ausführung und Zustandsübergänge
Modell-/Rollenauflösung und Ressourcenplanung	Verzweigungen, Schleifen und Workflow-Knoten
Agent Lifecycle und Priorisierung	Checkpointing und Wiederaufnahme
Globale Berechtigungen und Fehlerpolitik	Workflow-lokale Fehlerbehandlung
Ergebnisaggregation und Quality Gate	Human-in-the-loop innerhalb des Workflows

Startprofile

Agent	Verantwortung
Coding Orchestrator	Aufgaben zerlegen und delegieren
Code Reviewer	Diff, Architektur, Tests und Risiken prüfen
Research / Planning	Quellen bewerten, Optionen und Plan erstellen
Vision Specialist	Bilder und Dokumente auswerten
Embedding / Reranking	Memory-Index und Relevanzbewertung
Subtask Worker	Begrenzte, klar definierte Teilaufgaben ausführen

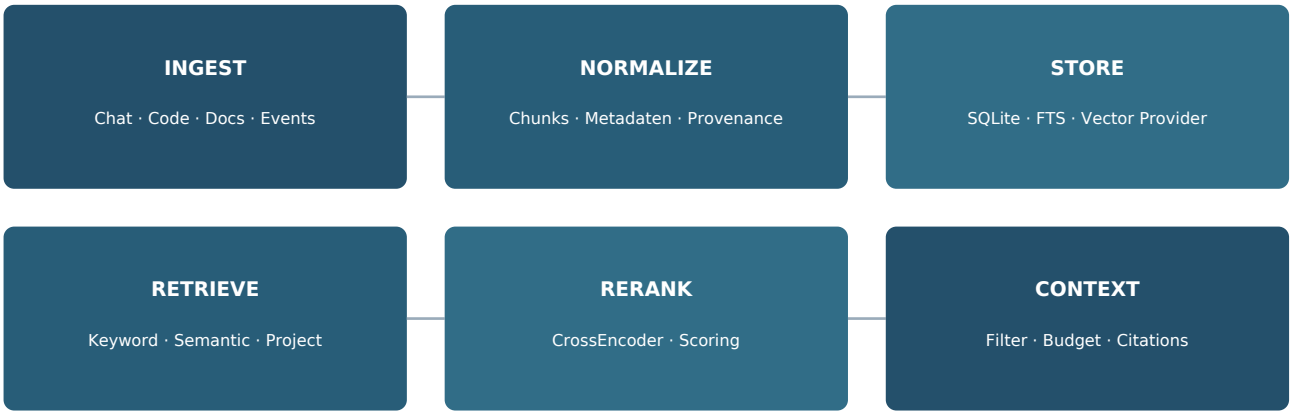
Jeder Agent besitzt einen einheitlichen Vertrag, Capability-Profil, Tool-Berechtigungen und Ressourcenbudget. Er ist nicht an ein konkretes Modell gebunden.

07 / MEMORY

Hybrid Memory: dauerhaft, suchbar und austauschbar

Workflow-Zustand, Gesprächskontext und langfristiges Projektwissen sind getrennte Datenarten.

MEMORY PIPELINE



Memory-Typ	Zweck
Working	Aktueller Task- und Gesprächskontext
Session	Sitzungsverlauf und offene Fragen
Episodic	Frühere Aufgaben, Ergebnisse und Fehler
Semantic	Wissensobjekte und Fakten
Project	Projektentscheidungen, Architektur, Codewissen
Procedural	Bewährte Abläufe und wiederverwendbare Routinen

Technologievorschlag

- SQLite + SQLAlchemy/Alembic als Metadaten- und Identitätsbasis.
- SQLite FTS5 für Volltextsuche; sqlite-vec oder FAISS als austauschbarer Vector Provider.
- sentence-transformers als Embedding-Kandidat; CrossEncoder für Reranking.
- LangGraph Checkpoints speichern Workflow-Zustand, ersetzen aber nicht das langfristige DBZS Memory.

Retrieval muss Herkunft, Aktualität, Projektzugehörigkeit und Vertrauensstatus berücksichtigen. Externe Inhalte sind Daten, keine Systemanweisungen.

08 / GUI & VISUAL DESIGN

Professionelle Entwickleroberfläche

Die bereitgestellte GUI-Richtlinie fordert Klarheit, kompakte Navigation, großzügige Arbeitsfläche und Funktion vor Dekoration.

Visuelle Leitlinien

Element	Richtlinie
Navigation	Klar, kompakt, logisch; Dashboard, Projekte, Modelle, Einstellungen, Logs
Arbeitsfläche	Großzügig, aufgeräumt, auf die aktive Aufgabe fokussiert
Seitenleisten	Nur relevante Informationen; kontextabhängig
Buttons & Icons	Eindeutige Aktionen, einheitliche und zurückhaltende Icons
Typografie & Farben	Technisch, gut lesbar, dezente funktionale Kontraste
Status	Backend, Modell, Runtime, Agent und Task eindeutig unterscheiden
Dialoge	Kompakt und zweckorientiert; keine unnötigen Effekte

Vorgeschlagenes Hauptlayout

DESKTOP WORKSPACE



Code-Intelligenz sichtbar machen

- Kontextpanel: betroffene Dateien, Symbole und Abhängigkeiten mit Herkunft.
- Task-Panel: Plan, aktuelle Aktion, Status, Freigabe und Fortschritt.
- Diff-Ansicht: Änderungen vor/nachher, Datei- und Zeilenbezug.
- Verification-Panel: Tests, Lint, Typprüfung mit tatsächlichem Status und Logzugriff.
- Runtime-Panel: Modellrolle, aktives Modell, Backend, Ressourcen und Swap-Status.

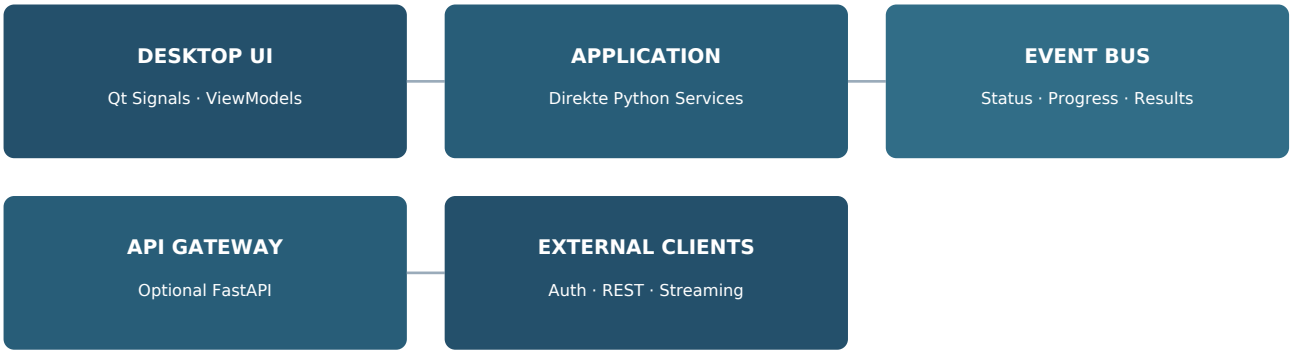
Wichtig: Das ist ein konzeptionelles Zielbild, kein Screenshot und kein Nachweis der bestehenden GUI-Implementierung. Keine bestehende Funktion darf zugunsten der Optik entfernt werden.

09 / APIS & PLATTFORM

Interne Services, optionale externe API, echte Cross-Platform-Architektur

Plattformneutralität wird über Adapter erreicht – nicht durch Annahme, dass jede Runtime auf jedem Gerät identisch funktioniert.

COMMUNICATION MODEL



Kommunikationsregeln

- Interne Kommunikation über Services und Contracts; kein HTTP-Zwang zwischen Modulen.
- Event Bus für Fortschritt, Status, Fehler und Task-Lifecycle.
- FastAPI nur bei Bedarf für externe Integrationen; standardmäßig lokal und geschützt.
- llama-server bleibt ein Inferenzprovider und ist nicht das API-Gateway der gesamten Anwendung.

Plattformadapter

Adapter	Verantwortung
Windows	Prozessoptionen, Pfade, GPU Provider, Packaging
Linux	Prozesssignale, Pfade, GPU Provider, Packaging
macOS	Metal/CPU-Pfade, Prozessmanagement, Packaging
Gemeinsamer Vertrag	Process Manager, Hardware Detector, Filesystem, Paths, Environment

Technologie-Baseline

Bereich	Technologie / Status
Sprache & Desktop	Python + PySide6/Qt – Baseline aus bisheriger Architektur
Inference	llama.cpp / GGUF – primärer lokaler Kandidat
Agenten	Eigener Orchestrator; LangGraph für komplexe Workflows
Persistence	SQLite, SQLAlchemy, Alembic
Contracts	Pydantic
Async / HTTP	asyncio, HTTPX; FastAPI optional extern
Memory	FTS5 + austauschbarer Vector Store + Embedding/Reranking Provider
QA / CI	pytest, pytest-qt, Ruff, mypy optional, Gitea Actions

10 / SICHERHEIT & KONTROLLE

Autonomie mit nachvollziehbaren Grenzen

Ein Coding-Agent braucht Zugriff auf Code und Werkzeuge – aber nicht unbegrenzte Ausführungsrechte.

Approval Levels

Level	Aktion	Policy
L0	Lesen und analysieren	Automatisch, workspace-begrenzt
L1	Vorschläge und Pläne	Automatisch, keine Seiteneffekte
L2	Dateien ändern	Explizite Policy / Freigabe
L3	Terminal und Prozesse	Kontrollierte Freigabe, Limits
L4	Commit / Push / Merge	Explizite Freigabe
L5	Systemweite Änderungen	Explizite, besonders deutliche Freigabe

Security-by-design

- Workspace-Boundaries und Path-Traversal-Schutz.
- Keine Secrets im Quellcode; ausschließlich Environment Variables.
- Externe Inhalte und Repository-Dateien als nicht vertrauenswürdige Daten behandeln.
- Prozess-, Netzwerk- und Ressourcenlimits für Execution.
- Audit-Logs und Evidence für relevante Aktionen.
- Keine destruktiven Änderungen, Installationen oder Remote-Aktionen ohne passende Autorisierung.

Evidence Contract

Jede relevante Ausführung wird mit Task-ID, Workspace-ID, Agent-ID, Aktion, Zeitstempeln, Status, Exit-Code, Artefakten und Approval-ID verknüpft. Dadurch lassen sich Ergebnisse später auf konkrete Ausführungen zurückführen.

Sandbox-Hinweis

Ein isolierter Python-Subprozess ist keine belastbare Sicherheits-Sandbox. Für nicht vertrauenswürdige Ausführung braucht es eine stärkere Isolation, zum Beispiel Container oder betriebssystemspezifische Sandbox-Mechanismen. Die konkrete Lösung ist plattformabhängig zu validieren.

11 / UMSETZUNG

Inkrementelle Roadmap und Quality Gates

Vor einem Refactoring wird der tatsächliche Repository-Zustand inventarisiert. Bestehende Implementierungen werden wiederverwendet, wenn sie die Zielverträge erfüllen.

Paket	Inhalt	Abnahme
AP-01 Core Foundation	Contracts, Events, Config, Service Boundaries, Platform Interfaces	Keine UI-/OS-Abhängigkeiten im Domain Core
AP-02 Runtime & Scheduler	Process Registry, Resource Estimator, Scheduler, Backend Adapter	Start/Stop/Health; Ressourcenprüfung; Fallback-Test
AP-03 Model Registry	Scanner, Metadata, Capabilities, MM Pairing, Role Picker	Modelle werden korrekt erkannt und gefiltert
AP-04 Orchestrator	Task Model, Agent Registry, Dispatcher, Aggregator	Task wird nachvollziehbar zugewiesen und abgeschlossen
AP-05 LangGraph	Adapter, Graph Registry, Checkpointing, Approval Nodes	Workflow kann pausieren und wiederaufgenommen werden
AP-06 Memory	FTS, Embeddings, Vector Provider, Reranking	Relevante Treffer mit Herkunft und Projektfilter
AP-07 Execution & API	Permissions, Evidence, Sandbox, optionales API Gateway	Nicht freigegebene Aktionen werden blockiert
AP-08 Cross-Platform	Adapter, Packaging, CI Matrix, Regression	Build-/Testnachweise je Zielplattform

Quality Gates

- Ruff: Lint und Format.
- mypy: Typprüfung dort, wo Typverträge etabliert sind.
- pytest: Unit- und Integrationstests.
- pytest-qt: GUI-Regressionen.
- Runtime Health Checks: Prozess, Backend, Modell und Ressourcen.
- Git-Diff Review: keine unbeabsichtigten Dateien oder Nutzeränderungen.
- CI: reproduzierbare Befehle und Ergebnisprotokolle.

Roadmap-Regel: Jede Phase liefert ein kleines, prüfbares Inkrement. Kein Big-Bang-Refactor und keine Behauptung von Tests ohne tatsächliche Ausführung.

12 / ENTSCHEIDUNGSKATALOG

Was ist entschieden – und was muss noch validiert werden?

Diese Trennung verhindert, dass Vorschläge später irrtümlich als bereits bestätigte Implementierungsfakten behandelt werden.

Kategorie	Inhalt
Vom Nutzer bestätigt	Eigener Orchestrator + LangGraph; dynamische Ressourcensteuerung; interne Services + externe API; Hybrid-Memory; Cross-Platform von Anfang an.
Aus GUI-Direktive übernommen	Funktion vor Dekoration; kompakte Navigation; großzügiger Arbeitsbereich; klare Statusanzeigen; keine unnötigen visuellen Effekte; bestehende Funktionen erhalten.
Technische Baseline aus bisheriger Diskussion	Python, PySide6/Qt, llama.cpp/GGUF, SQLite, Pydantic, pytest, Git und Gitea CI.
Vorgeschlagene Erweiterungen	AST-/Symbolindex, Code Graph, Hybrid Code Retrieval, Diff-basierter Editierzyklus, Evidence-gebundene Verifikation, Capability & Provider Registry.
Noch zu validieren	Python-/Qt-Kompatibilitätsmatrix, konkrete Vector-Store-Wahl, GPU-Backend-Matrix, LangGraph Checkpointer, Sandbox pro OS, Packaging.
Nicht behauptet	Dass alle Zielkomponenten bereits existieren, alle Plattformen getestet sind oder die vorgeschlagene Zielstruktur schon umgesetzt wurde.

Architektur-Entscheidungen (ADR) – initiale Liste

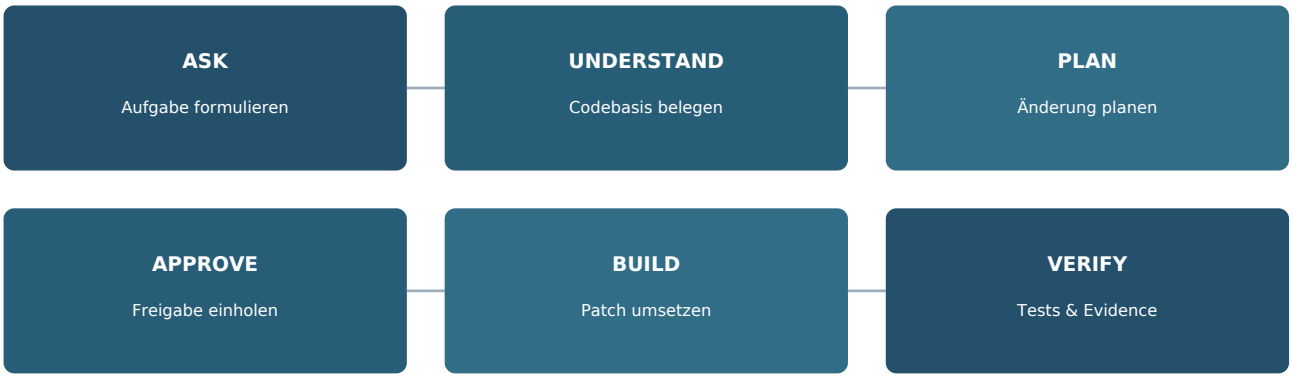
ADR	Entscheidung
ADR-001	Core bleibt unabhängig von UI- und Provider-Frameworks.
ADR-002	DBZS Orchestrator besitzt globale Task-Steuerung; LangGraph führt komplexe Workflows aus.
ADR-003	Alle Modellstarts laufen über zentralen Runtime Controller und Scheduler.
ADR-004	Modelle, Capabilities, Rollen und Runtime-Profile sind getrennte Entitäten.
ADR-005	Memory Provider sind austauschbar; SQLite bleibt Metadatenbasis.
ADR-006	Interne Services sind direkt; externe API ist optional.
ADR-007	Plattformabhängige Funktionen liegen in Adaptern.
ADR-008	Codeänderungen sind diff-basiert, freigabegesteuert und evidence-gebunden.

13 / ZIELBILD

Definition of Done – DBZS Code Intelligence

Das Produktziel ist erreicht, wenn Codeverständnis und kontrollierte Ausführung als durchgängiger Arbeitsprozess funktionieren.

END-TO-END DEVELOPER EXPERIENCE



Definition of Done

- DBZS kann ein Repository inventarisieren und die aktive Workspace-/Git-Identität erfassen.
- Code-Suche kombiniert Text-, Symbol- und semantische Treffer.
- Ein Task kann relevante Dateien, Symbole, Tests und Abhängigkeiten belegen.
- Der Orchestrator kann Aufgaben passend zu Rolle, Modell und Ressourcen delegieren.
- Komplexe Workflows können pausieren, fortgesetzt und kontrolliert freigegeben werden.
- Änderungen werden als nachvollziehbarer Diff erstellt; bestehende Nutzeränderungen bleiben geschützt.
- Tests und Qualitätsprüfungen werden tatsächlich ausgeführt oder als nicht ausgeführt markiert.
- Ergebnis, offene Risiken und Evidence werden in einem verständlichen Abschlussbericht geliefert.

Abschließende Leitlinie

DBZS-CODING-CHAT ist kein Chatfenster mit zusätzlichen Buttons. Es ist eine lokale Entwicklungsplattform, deren Intelligenz aus dem Zusammenspiel von Codeverständnis, Modellkompetenzen, Ressourcensteuerung, Agentenkoordination, Memory, Werkzeugen und überprüfbarer Ausführung entsteht.

ANALYSIEREN. BELEGEN. PLANEN. FREIGEBEN. UMSETZEN. VERIFIZIEREN. LIEFERN.

Quellenbasis und Einordnung

Dieses Blueprint konsolidiert die Architekturentscheidungen aus dem Chat vom 02.11.2026, die bereitgestellte GUI Design Directive für DBZSCC2611, die Modellzuordnungs-/Model-Registry-Notizen sowie die im Projektkontext verfügbaren DBZSCC/AIS2-Strategie- und Implementierungsnotizen. Der Inhalt unterscheidet bestätigte Entscheidungen, technische Baselines und neue Vorschläge. AIS2 bleibt ein separates Produkt-/Visionsthema und wird nicht stillschweigend mit DBZS-CODING-CHAT zusammengeführt.

Dokumentende · DBZS DEVCØRE